

Embedded Systems 2WD and 3WD Vehicle Design

Aidan Donnellan

June 2026

1 Introduction

Throughout Phase D of Embedded System programming, we have focused on the start-to-finish evolution of a 2 wheel drive robot. Throughout the design process, we utilized tools such as KiCAD, STM32IDE and STM32programmer, OnShape, and others to flesh out the entire system. External peripherals such as the MPU6050 IMU and ESP8266 wifi module were used to support features of the system. This report aims to break down each design phase of the robot from PCB design to final code.

2 Design Stage

The design state of the 2 wheel drive balancing robot began with a block diagram of the system. This system blueprint defined all power domains, communication lines, and signal routing between the STM32, sensors, and motor circuitry. The block diagram serves as an absolute reference point for all subsequent hardware implementation and pin mapping.

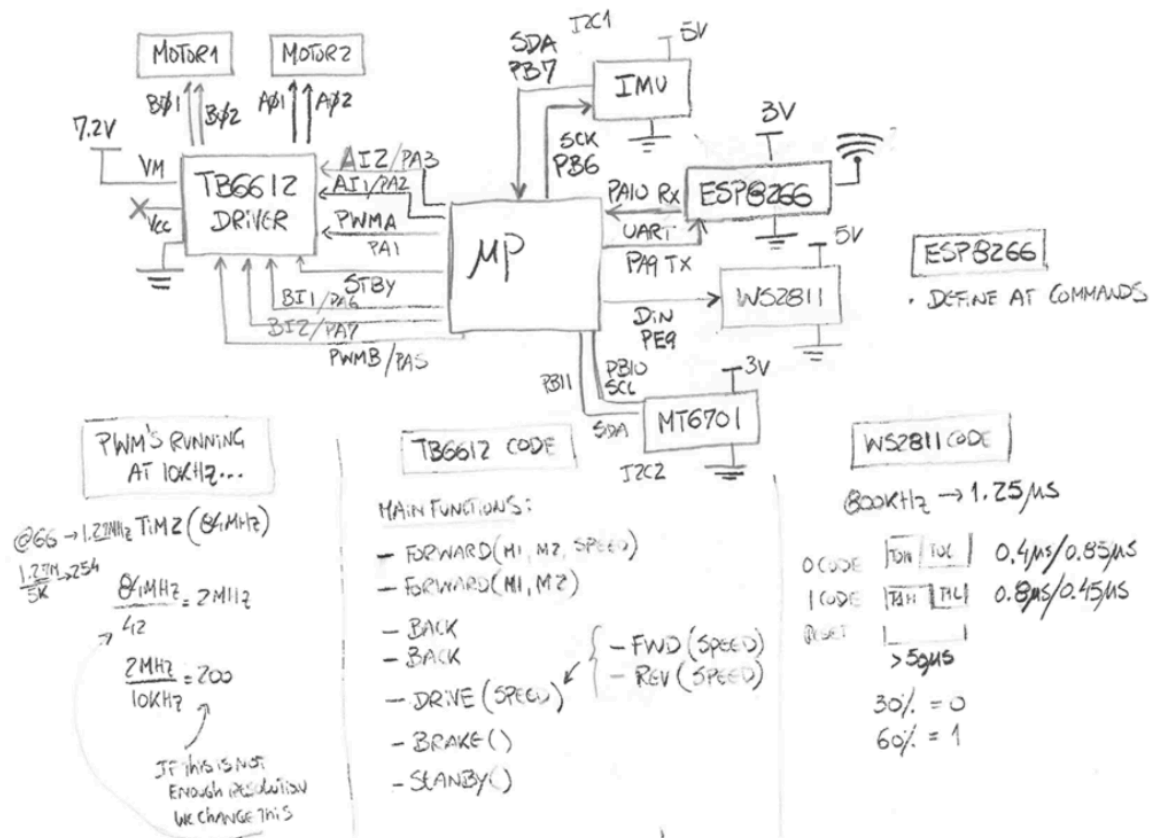


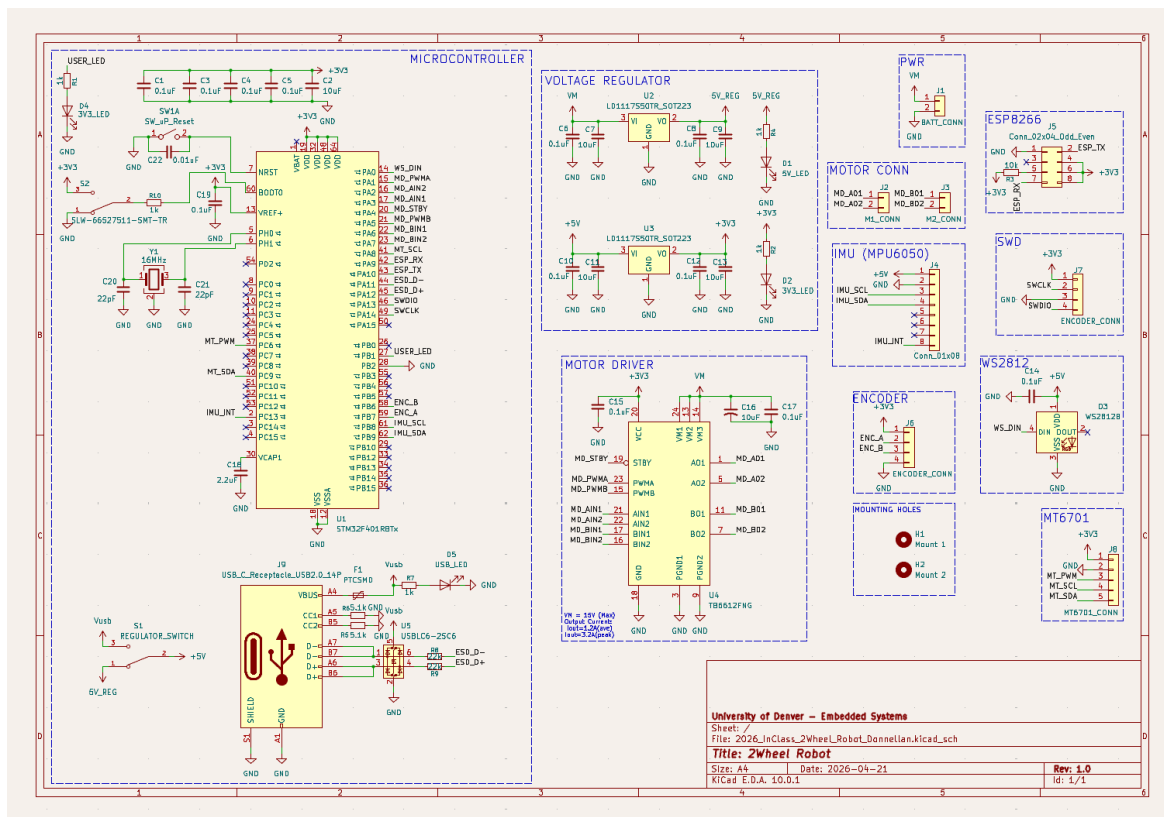
Figure 1: Block Diagram Created by Dr. Goncalo Martins

3 PCB Design and Manufacturing

This section aims to cover the complete start to finish of the PCB design and manufacturing stage of the board.

3.1 Schematic

The schematic was designed in KiCAD and converted the fundamental information from the block diagram into the circuit layout. This stage involved configuring the decoupling capacitor networks for the STM32F401 microprocessor, setting up current-limiting resistors for status LEDs, and pull-up resistors for I2C communication. Each sensor, motor, and circuit section is broken down into individual blocks on the schematic.



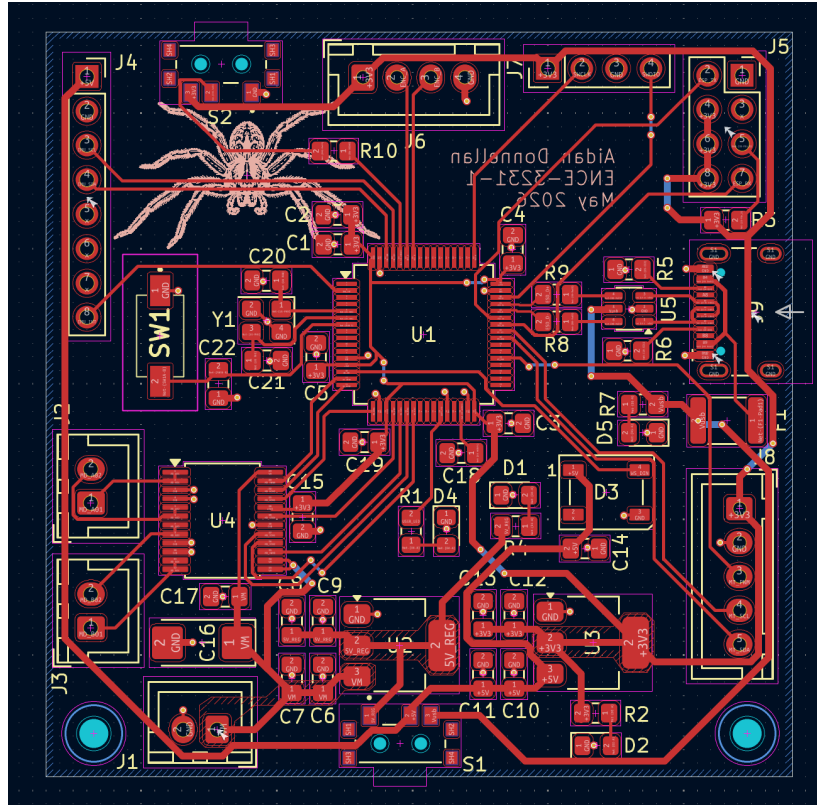


Figure 3: KiCAD PCB Assembly Layout

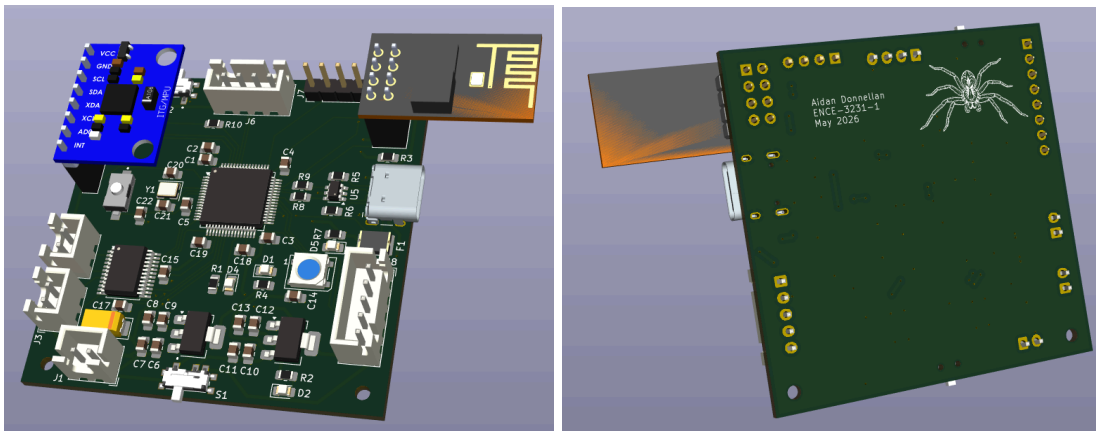


Figure 4 and 5: 3D Front and Back PCB Render

3.3 Assembly Stage

Upon receiving the PCB boards from PCBway, the physical assembly and solder phase commenced. Components were populated starting with the STM32F401 microprocessor and motor driver. Dr. Goncalo Martins proceeded to solder other components such as the Adafruit Neopixel LED and crystal oscillator before the board was returned and all other components such as SMT resistors and capacitors were soldered. Finally any through hole header and socket pins were soldered to the board.

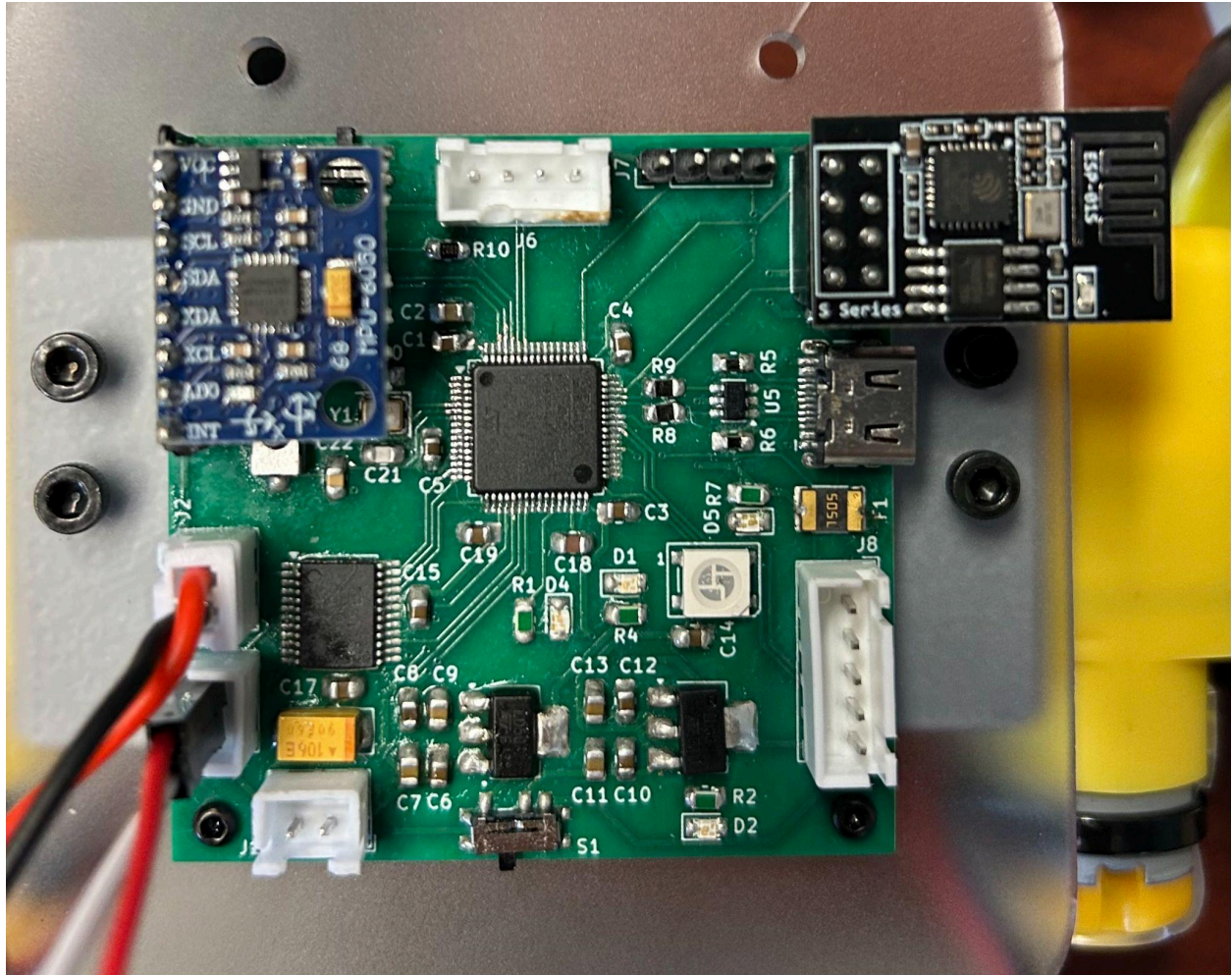


Figure 6: Fully Assembled Board Integrated with ESP-8266 Wifi Module and MPU6050 IMU

4.3 Wheel Robot and Chassis Design

A structural chassis was designed for the system on OnShape and fabricated using acrylic. The design incorporated a region for a third caster wheel in order to first test the system as a 3WD vehicle. This provides the system with a controlled environment to verify wheel encoder feedback, wireless telemetry channels, and motor driver responsiveness.

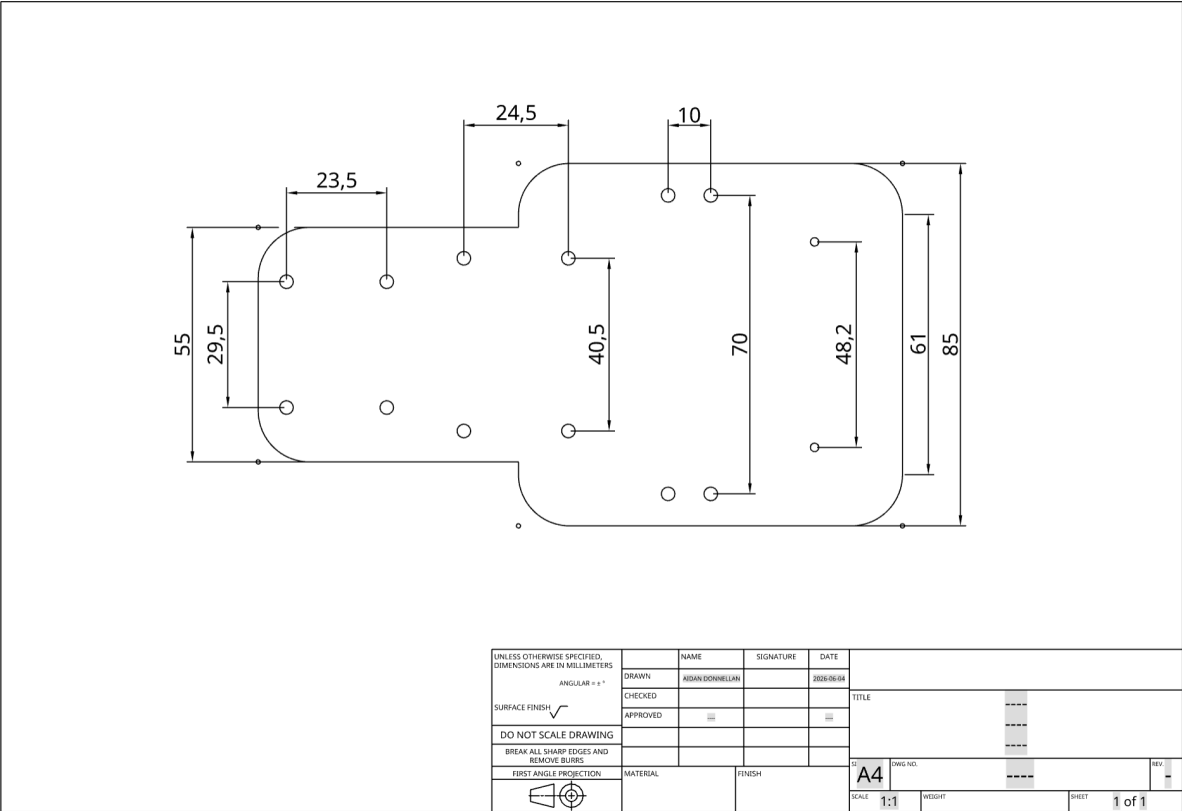


Figure 7: 3WD Chassis Design Created on OnShape

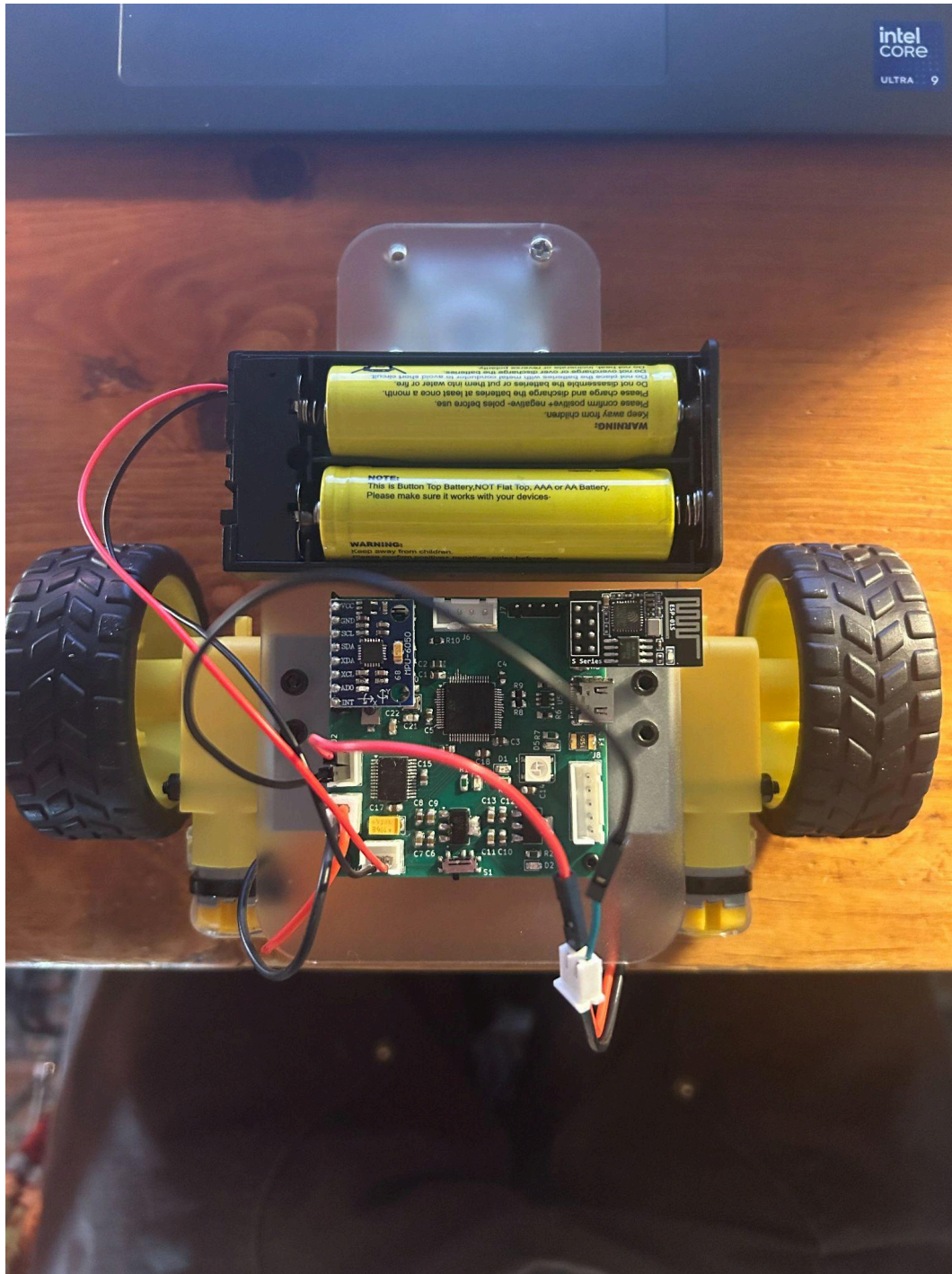


Figure 8: Fully Assembled 3WD Vehicle

5 Programming and Testing the 3 Wheel Robot

Initial hardware verification of the system encountered immediate software integration challenges when deploying the baseline 3WD code file written by Dr. Goncalo Martins. Although

the code compiled successfully through STM32CubeProgrammer, the drive motors failed to rotate.

A troubleshooting process then ensued and was done to isolate any failure points. Probing the onboard motor driver IC with a digital multimeter confirmed the motor driver chip was receiving the primary 7.4V; however, the output channels delivered a small 25-30 mV signal when actuation commands were active. Further continuity testing and signal tracing revealed the motor driver was not receiving the essential 3.3V logic signals from the microcontroller to power the PWM duty cycles for the motors. The board was brought to Dr. Goncalo Martins, where the primary STM32F401 microcontroller was desoldered and replaced with a new IC. This hardware intervention fixed the logic issues and restored the full signal.

Before uploading an updated 3WD code file in STM32CubeProgrammer, the development environment required certain configuration adjustments within the Linux operating system. USB device permission constraints initially blocked connection to the ST-Link debugger interface. This was quickly resolved by modifying the systems udev rules to grant read and write permissions.

Once resolved, the firmware was flashed successfully, and the battery pack was initialized. The 3-wheel robot responded accurately to the remote control commands, and transmitted real time data from the remote control created by Dr. Goncalo Martins. This included speed, roll, pitch, and counter. The vehicle could then be controlled via manual arrow key inputs. This development allowed for moving toward a 2 wheel self-balancing system.

6 2 Wheel Robot Testing

Transitioning the vehicle to a dynamic, self balancing 2-wheel configuration introduced a significant electrical stress challenge. A PID control algorithm was developed with the help of Google Gemini pro and integrated directly into the core firmware infrastructure. To establish a true vertical reference, Google Gemini Pro assisted with interfacing the board to an Arduino IDE serial port via a virtual COM port that read the processed MPU6050 IMU values. Once the stable angular data was verified, systematic tuning of the proportional (Kp), integral (Ki), and derivative (Kd) feedback gains were conducted to maximize restoring forces and minimize overshoot.

During testing, severe mechanical and electrical anomalies emerged. The center mass of the physical assembly was not as optimal as anticipated, with the heavy battery pack positioned too far back on the chassis, creating a large destabilizing moment that caused the robot to overshoot and roll backward past a recoverable pitch limit.

This mechanical imbalance led to a critical electrical failure during a high-tilt recovery attempt. As the PID controller responded to a high pitch error, it drove the motor PWM cycles to a 100% duty cycle. Because the motors stalled by the weight of the chassis, the system drew excess current, exceeding the thermal capacity of the power wiring, and causing the power wiring on the battery pack to melt.

Fortunately, the PCB's layout was protected against this spike, leaving the central STM32 and sensor arrays undamaged. This incident highlighted a severe vulnerability and misjudgement in running the raw high duty cycle PWM signals during motor stall without software current-limiting safety thresholds. While a brief sequence of self-balancing behavior was exhibited, a long-term reliable system requires restructuring the physical layout to lower the center of gravity and implementing software protection algorithms to handle a current spike.

7 Documentation

- ENGREdu By Dr. Goncalo Martins:
<http://engredu.com/2026/05/01/2-wheel-balance-robot/>
- KiCAD
- OnShape
- Google Gemini Pro for Code Assistance